Encuestas de Satisfacción Docente: Tu Opinión Transforma la US

Las encuestas de satisfacción son la vía oficial e institucional para que el estudiantado valore la docencia en la Universidad de Sevilla. A través de un **proceso 100% anónimo,** los resultados impulsan la mejora continua del profesorado, el programa DOCENTIA y la acreditación oficial de las titulaciones.

CONFIDENCIALIDAD ABSOLUTA



Autenticación con UVUS solo para verificar legitimidad

Tu usuario valida que puedes participar, exactamente igual que identificares antes de votar.



Respuestas 100% anónimas

En la Secretaria Virtual, tu identidad jamás queda vinculada a tus respuestas emitidas.



Informes globales tras cerrar actas

El profesorado solo recibe datos agrupados **y anónimos** una vez finalizada la evaluación.

https://sevius4.us.es/?valorus

UTILIDAD Y EFECTIVIDAD DE PARTICIPAR



Impacto directo en la calidad docente y programa DOCENTIA

Base para la formación del profesorado, promociones y reconocimiento de la excelencia.



Acreditación oficial de los títulos

Tus valoraciones son esenciales para respaldar los Sistemas de Garantia de Calidad oficiales.



Solo requiere 5 minutos

Un proceso ágil y sencillo para construir de forma activa la universidad del futuro.

Gemini Notebook

Diccionarios y tablas hash

Buscar por clave más deprisa que un árbol

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Agenda de la clase

1. **El TAD Diccionario**, y lo que ya sabemos hacer con él
2. **Funciones hash**: de la clave a la casilla
3. **Encadenamiento**: una lista por cubeta, y el rehash
4. **Direccionamiento abierto**: todo dentro del array, y las lápidas
5. **La STL**: unordered_map y unordered_set
6. **El trie**: un árbol de letras para claves de texto
7. **Árbol, tabla hash y trie**, frente a frente

De dónde venimos

Del tema 2:

La lista enlazada: vuelve como **cadena** de cada cubeta
El **coste amortizado** del array dinámico: vuelve tal cual en el **rehash**

Del tema 4:

El **BST** y **std::map**: un diccionario que busca en O(log n), siempre
El **árbol general**: vuelve como **trie**
El **montículo**: vuelve en las prácticas para el top-k

La pregunta del tema: ¿se puede buscar más deprisa que log n?

El hilo conductor del tema

Bajar de O(log n) se puede. Pero cada paso cobra algo a cambio:

Bloque	Lo que aporta	Lo que cuesta
Direccionamiento directo	O(1) exacto: la clave es el índice	Memoria del universo de claves
Función hash	Cualquier clave → una de m casillas	Colisiones, inevitables
Encadenamiento	Una lista por cubeta: O(1 + α)	Un nodo por clave; rehash
Direccionamiento abierto	Todo en el array: caché	α < 1, agrupamiento, lápidas
Trie	Claves de texto: O(L) y prefijos	Mucha memoria por nodo

Lo que pierde la tabla hash para ganar velocidad: **el orden**.

El TAD Diccionario

Pares (clave, valor), y se usa por clave

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

El TAD Diccionario

Pares **(clave, valor)** sin claves repetidas. Nadie navega por él: se pide **por clave**.

Operación	**Postcondición**
insertar(c, v)	c tiene asociado v; si ya estaba, su valor se **sustituye**
buscar(c, v)	devuelve si c está; si está, deja su valor en v
borrar(c)	si c estaba, la quita y devuelve true
tamanio()	número de claves distintas

Sin valor, solo claves, es un **Conjunto** (set, unordered_set).
Ya conoces dos: el BST del tema 4 era un Conjunto, y std::map un Diccionario.

01_tad_diccionario.cpp

Recordatorio

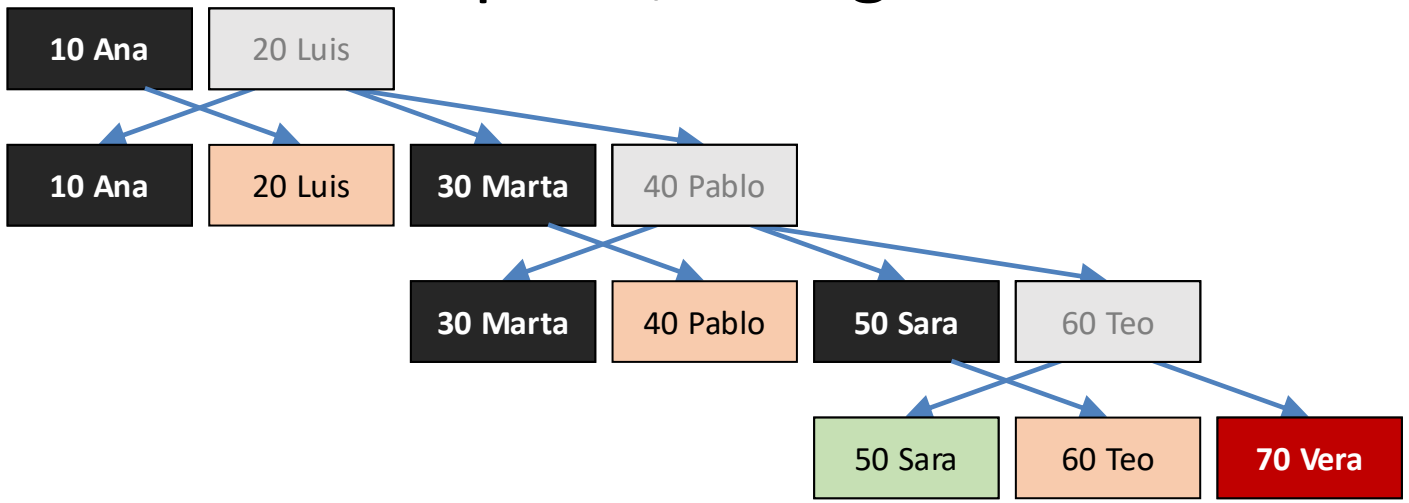
El diccionario del tema 4

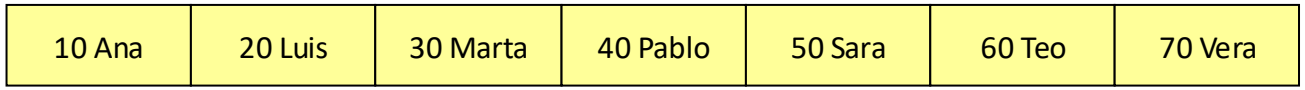
```
std::map<int, std::string> alumno;
alumno[10] = "Ana";
alumno[20] = "Luis";
alumno[30] = "Marta";
alumno[40] = "Pablo";
alumno[50] = "Sara";
alumno[60] = "Teo";
alumno[70] = "Vera";

auto it = alumno.find(50);
for (auto& [k, v] : alumno)
   std::cout << k << " " << v;
```

Cada nodo guarda el PAR (clave, valor)
y se ordena solo por la clave.
O(log n) siempre, y recorrido ORDENADO:
justo lo que la tabla hash no tendrá.

std::map<int, string>





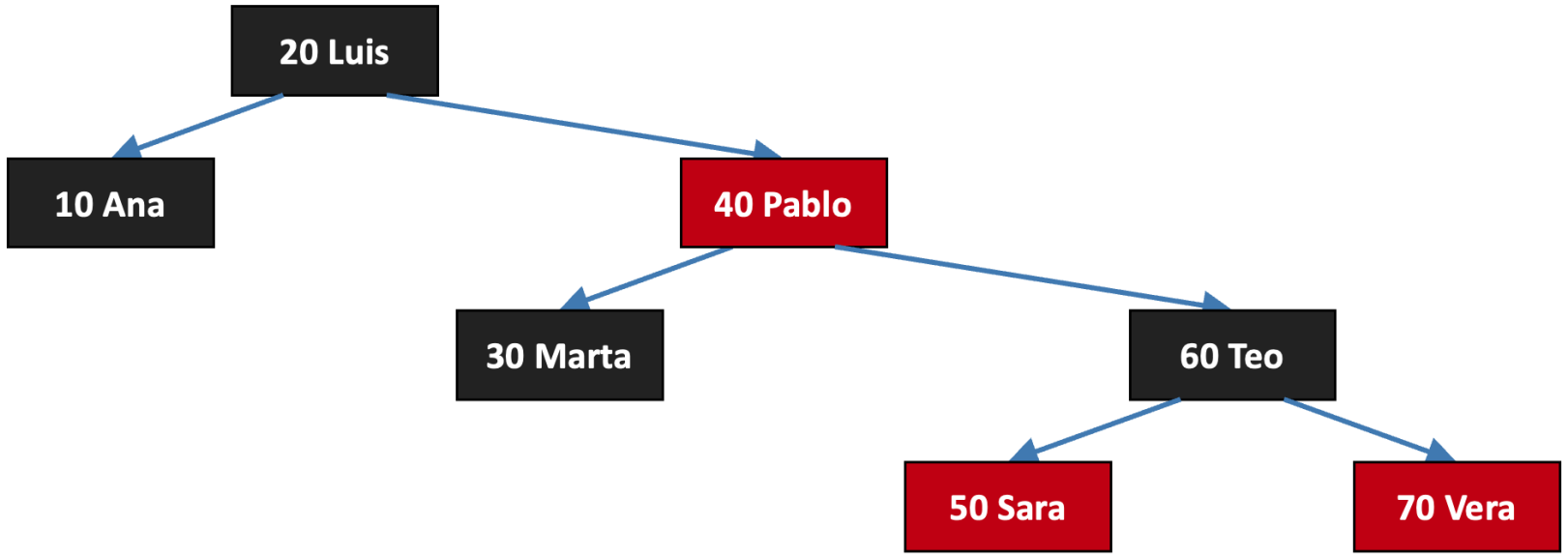
recorrido ordenado por clave

Recordatorio

El diccionario del tema 4

```
std::map<int, std::string> alumno;
alumno[10] = "Ana";
alumno[20] = "Luis";
alumno[30] = "Marta";
alumno[40] = "Pablo";
alumno[50] = "Sara";
alumno[60] = "Teo";
alumno[70] = "Vera";

auto it = alumno.find(50);
for (auto& [k, v] : alumno)
    std::cout << k << " " << v;
```



Cada nodo guarda el PAR (clave, valor)
y se ordena solo por la clave.
O(log n) siempre, y recorrido ORDENADO:
justo lo que la tabla hash no tendrá.

Lo que ya sabemos hacer

Implementación	buscar	insertar	borrar
Vector sin ordenar	O(n)	O(n): ¿ya estaba?	O(n)
Vector ordenado	O(log n)	O(n): desplazar	O(n)
std::map (rojo-negro)	O(log n)	O(log n)	O(log n)

Medido: insertar n claves y buscarlas todas (ms)

n	sin ordenar	ordenado	std::map
1 000	2,2	0,5	0,3
4 000	14,7	5,0	1,3
16 000	395,3	66,4	5,9

01_tad_diccionario.cpp

Implementación

Direccionamiento directo

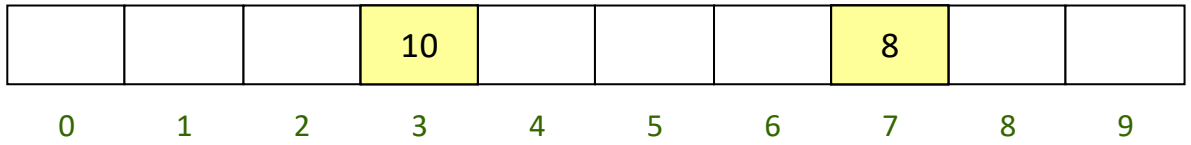
notaDe[clave]

```
// claves: numero de lista, 0..9
std::vector<int> notaDe(10, -1);

notaDe[3] = 10;   // insertar
notaDe[7] = 8;
int n = notaDe[7];   // buscar
notaDe[3] = -1;  // borrar
```

Ni comparaciones ni recorridos:
una operación de array. O(1) exacto.

Pero el array mide lo que mide el
UNIVERSO de claves posibles.



buscar(7) no [illegible] una posición

La letra pequeña: el universo de claves

¿Y si la clave es un **DNI** (8 dígitos) y hay 300 alumnos?

Casillas necesarias (el universo entero)	**100 000 000**
Memoria, a 4 bytes por casilla	381 MB
Casillas usadas	300 (0,0003 %)
Y si la clave es una cadena de 10 letras	$26^{10} \approx 1{,}4 \cdot 10^{14}$ casillas

¿Qué tendría que hacer una función que llevara cada DNI a una de 600 casillas?
¿Qué es inevitable que pase?

Ser **determinista** y **repartir** bien. Y es inevitable que dos DNIs caigan en la misma casilla. Esa función es una **función hash**; los choques, **colisiones**.

01_tad_diccionario.cpp

Funciones hash

De la clave a la casilla

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Qué se le pide a una función hash

`h : claves → {0, 1, …, m−1}` (m = número de cubetas)

1. **Determinista**: la misma clave da siempre la misma casilla.
   Si no, lo guardado no se vuelve a encontrar.

2. **Rápida**: O(longitud de la clave), nada más.

3. **Que reparta**: claves parecidas (casa, cosa, caso) o con patrón
   (todas múltiplos de 10) deben ir a casillas distintas.

Lo que **no** se le puede pedir: que no haya colisiones.
Hay más claves posibles que casillas.

02_funciones_hash.cpp

Implementación

Funciones hash

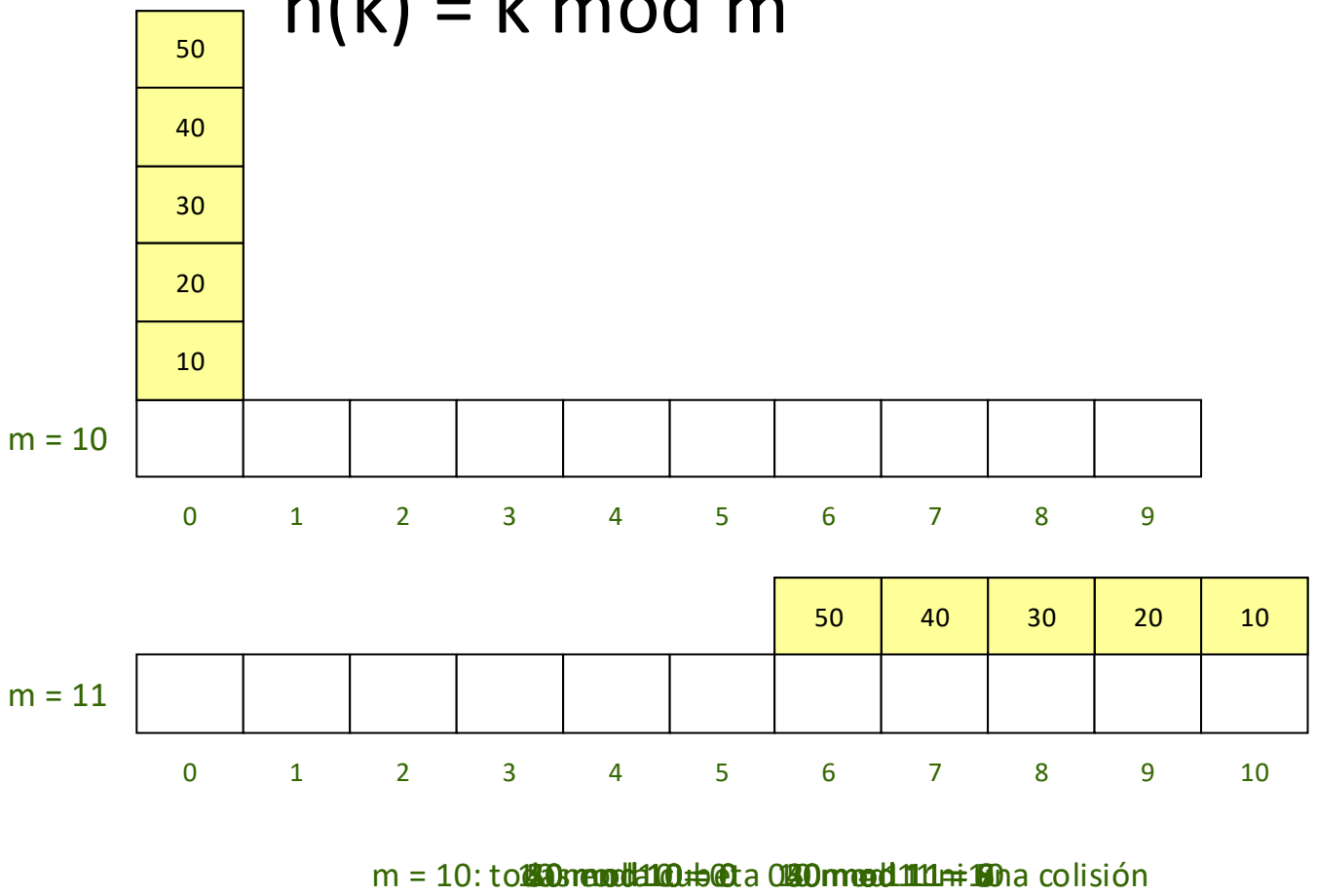
```
int hashModulo(long long k, int m) {
    return k % m;
}
```

Si las claves tienen un patrón que comparte un factor con m, casi todas las cubetas se quedan vacías.

Un m PRIMO no comparte factores con ningún patrón: las usa todas.

Medido con 1000 múltiplos de 10:
m = 100 usa 10 cubetas; m = 101, 101.

h(k) = k mod m



m = 10: todas [illegible] una colisión

Claves cadena: tres candidatas

Función	Idea	Defecto
Longitud	s.size()	Hay muy pocas longitudes distintas
Suma	Suma de los códigos de los caracteres	Ignora el ORDEN (anagramas), rango pequeño
Polinómica	Cada carácter pesa según su POSICIÓN	—

Medido: 50 000 palabras en m = 10 007 cubetas ($\alpha = 5$)

Función	Cubetas usadas	Máx. por cubeta	Comparaciones
Longitud	11 (0,1 %)	11 249	3 676
Suma	933 (9,3 %)	234	50,9
Polinómica	9 949 (99,4 %)	15	**3,50**

Reparto perfecto: $1 + \alpha/2 =$ **3,50**. El polinómico lo clava.

02_funciones_hash.cpp

¿De dónde sale 1 + α/2?

Suponemos que las claves $x_1, x_2, \ldots, x_n$ se insertan en ese orden y que cada una se añade **al final** de su lista. Con inserción al principio sale lo mismo por simetría.

Cuando se busca x_i, las claves que tiene delante son las que se insertaron antes que ella (hay i–1) y cayeron en su misma cubeta. Cada una cae allí con probabilidad 1/m, así que el número esperado de claves por delante es:

$$\frac{i-1}{m}$$

El coste esperado de buscar x_i es entonces 1 + (i–1)/m. Promediando sobre todas las claves, que tienen la misma probabilidad de ser buscadas:

$$\frac{1}{n}\sum_{i=1}^{n}\left(1+\frac{i-1}{m}\right) = 1+\frac{1}{nm}\sum_{i=1}^{n}(i-1) = 1+\frac{1}{nm}\cdot\frac{n(n-1)}{2} = 1+\frac{n-1}{2m}$$

$$= 1+\frac{\alpha}{2}-\frac{\alpha}{2n} \approx 1+\frac{\alpha}{2}$$

El término α/(2n) es despreciable, y el coste total se suele escribir Θ(1 + α).

Implementación

Funciones hash

```
unsigned long long
hashPolinomico(const std::string& s) {
   unsigned long long h = 0;
   for (char c : s)
      h = h * 31 + (unsigned char)c;
   return h;
}
```

h("sol") = 's'·31² + 'o'·31 + 'l'
en una sola pasada: regla de HORNER.

$108+111*31+115*31^2$ misma operación pero más eficiente:
https://es.wikipedia.org/wiki/Algoritmo_de_Horner
n sumas y $(n^2+n)/2$ multiplicaciones vs
n sumas y n multiplicaciones

Una letra en otra posición da otro hash: los anagramas ya no chocan.

hashPolinomico("sol")

h	carácter	h * 31 + carácter
0	's' = 115	115
115	'o' = 111	3676
3676	'l' = 108	114064
114064		

cubeta = 114064 mod 7 = 6

el mismo cálculo 3676 = 31*0+108 = 114064 denamiento.cpp

Ejercicio 2: el hash polinómico

Completa hashPolinomico() con la regla de Horner y B = 31.

Con palabras de más de 13 letras el resultado se sale de cualquier entero.
¿Es un error? ¿Por qué unsigned long long y no long long?

No es un error. Con **unsigned**, C++ garantiza aritmética **módulo 2^{64}**: el valor da la vuelta, pero siempre igual, y el hash sigue siendo determinista.

Con signo, desbordar es **comportamiento indefinido**.
El (unsigned char)c existe por lo mismo: una ñ o una á son char negativos.

02_funciones_hash.cpp

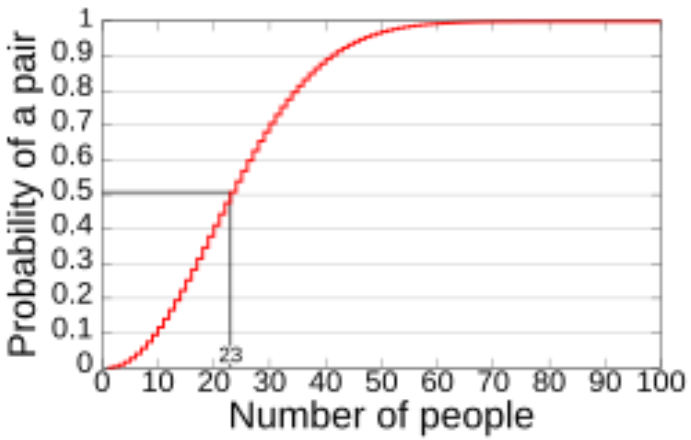
Las colisiones llegan pronto

Metiendo claves al azar en m cubetas, ¿cuántas hasta la **primera** colisión?
La intuición dice m/2. La realidad: **√(πm/2)**, [la paradoja del cumpleaños](#).

m	**√(πm/2)**	**medido**	**m/2**
365	23,9	24,5	182
10 007	125,4	129,5	5 003
1 000 003	1 253,3	1 152,7	500 001

Con un millón de cubetas, la primera colisión llega con la tabla al **0,1 %**.

No se pueden evitar las colisiones: hay que resolverlas. Dos maneras.



02_funciones_hash.cpp

Encadenamiento

Una lista enlazada por cubeta

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Encadenamiento

Cada cubeta guarda una **lista enlazada** (la del tema 2) con las claves que caen en ella.

insertar: h(c), recorrer esa lista por si ya estaba, y si no, **al principio**: O(1)
buscar / borrar: recorrer solo esa lista

El coste es la longitud de la lista. La mide el **factor de carga**:

$\alpha = n / m$ (claves por cubeta, de media)

buscar con éxito $\approx 1 + \alpha/2$ comparaciones buscar sin éxito $\approx \alpha$

Si m es fijo y n crece, α crece con n: **buscar vuelve a ser O(n)**.

03_hash_encadenamiento.cpp

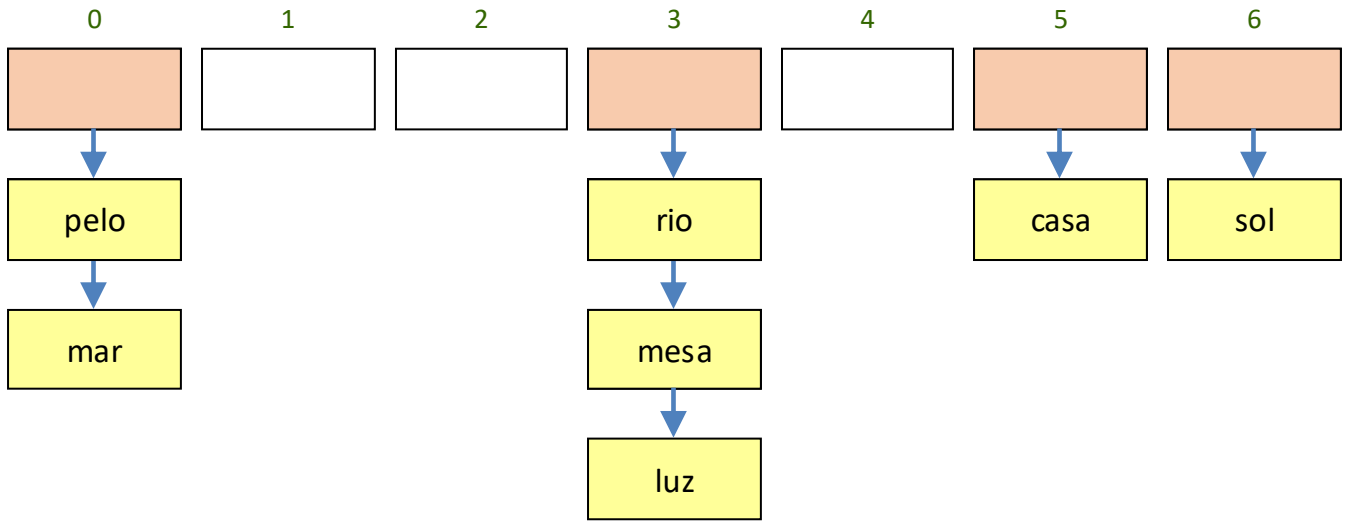
Implementación

```
struct Nodo {
    std::string clave;
    int valor;
    Nodo* siguiente;
};
```

Encadenamiento

```
void insertar(const string& c, int v) {
   int i = hash(c) % m;
   for (Nodo* p = cubetas_[i]; p; p = p->siguiente)
      if (p->clave == c) {
         p->valor = v; return;
      }
   cubetas_[i] = new Nodo{c, v, cubetas_[i]};
   n_++;
}
```

insertar(c, v)



Insertar al PRINCIPIO de la lista:
el insertarPrincipio del tema 2, O(1)
(tras mirar si ya estaba). Ojo: insertamos clave + valor + siguiente (nodo)

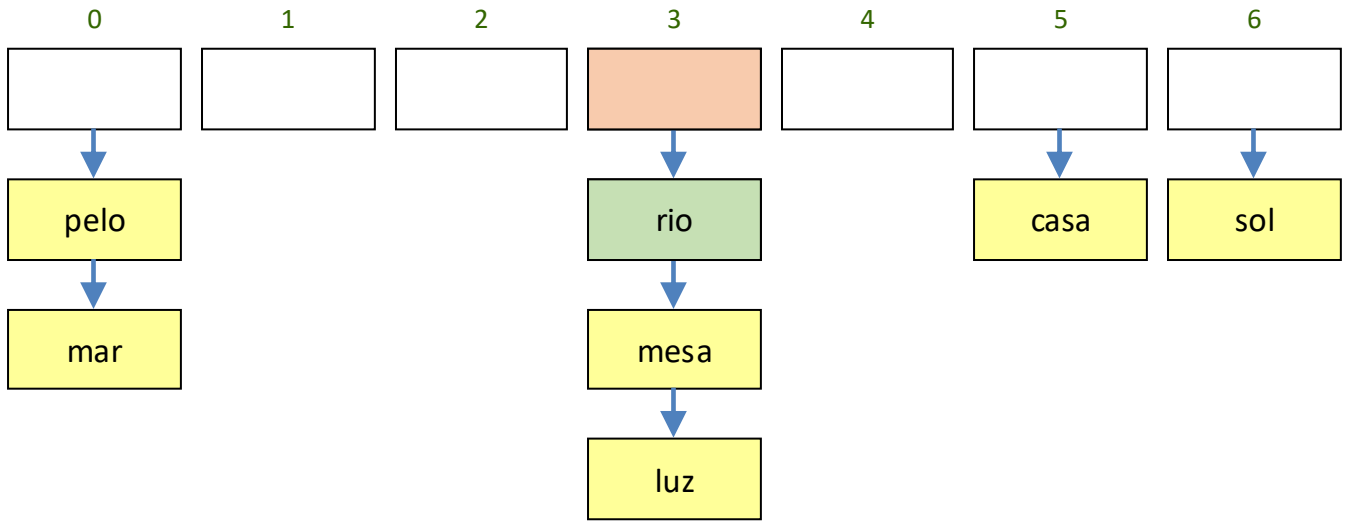
Implementación

Encadenamiento

```
bool buscar(const string& c, int& v) {
    int i = hash(c) % m;
    for (Nodo* p = cubetas_[i]; p;
         p = p->siguiente)
        if (p->clave == c) {
            v = p->valor; return true;
        }
    return false;
}
```

Solo se mira UNA lista, no la tabla.
Coste: lo larga que sea esa lista,
que de media es α.

buscar("rio")



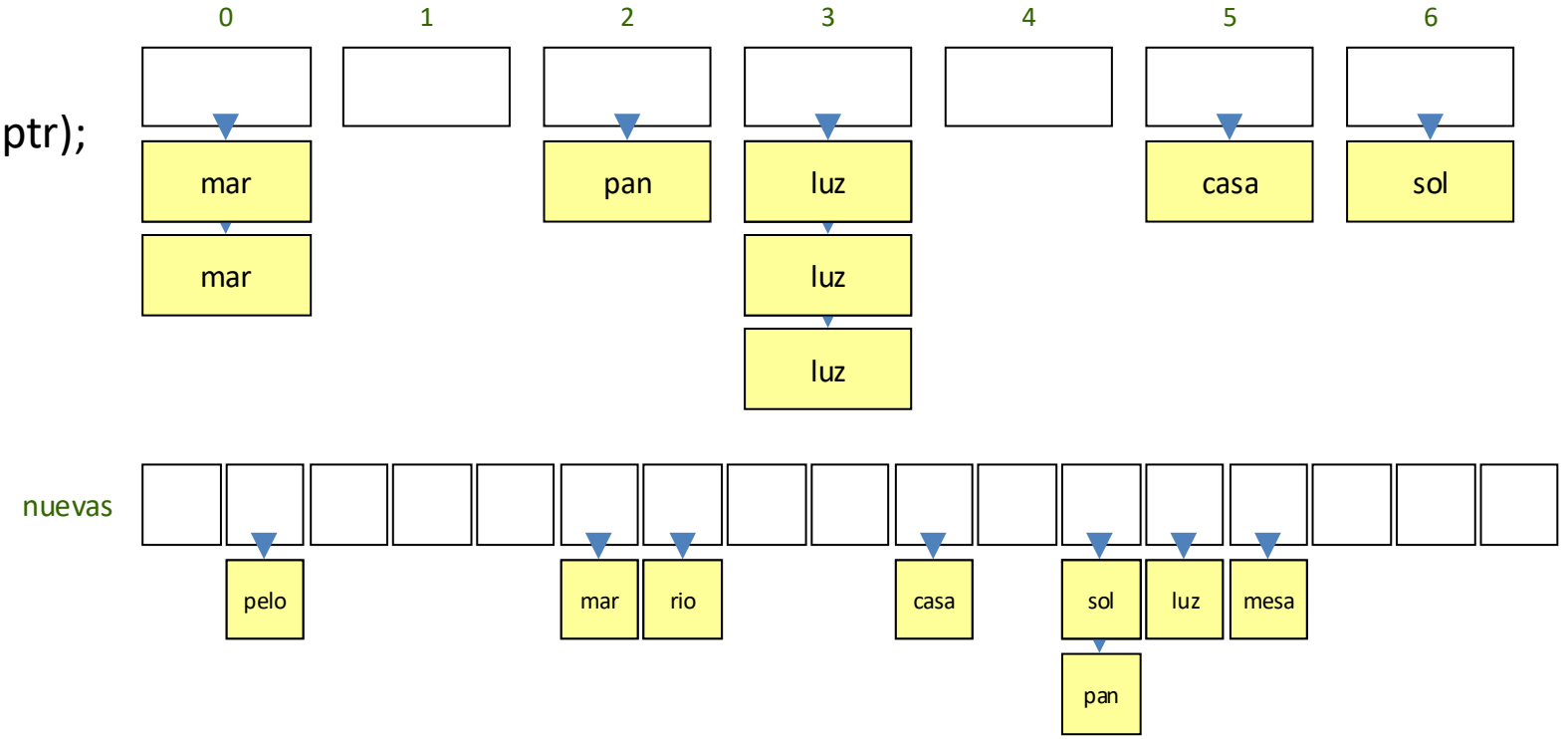
comparaciones

Implementación

Encadenamiento

```
void rehash(int nuevoM) {
  vector<Nodo*> nuevas(nuevoM, nullptr);
  for (Nodo* p : cubetas_)
    while (p != nullptr) {
      Nodo* sig = p->siguiente;
      int j = hash(p->clave) % nuevoM;
      p->siguiente = nuevas[j];
      nuevas[j] = p;
      p = sig;
    }
  cubetas_.swap(nuevas);
}
```

rehash(17)



Ni un new ni un delete de Nodo: se REENLAZAN. Y cada clave va a su NUEVA cubeta: hash % m cambia con m.

insertar(pan) → hash(14) = 17

Ejercicio 3: el rehash

Completa rehash(nuevoM) en 03_hash_encadenamiento.cpp.

¿Por qué no basta con copiar las cubetas viejas al principio del array nuevo?
¿Hace falta algún new o delete de Nodo?

Porque **hash % m cambia con m**: copiada sin más, una clave se buscaría en otra cubeta y no se encontraría. Cada una tiene que ir a **su nueva** cubeta.

Ningún new ni delete: los nodos ya existen, solo se **reenlazan**.
Cuidado con guardar p->siguiente **antes** de reenlazar p.

03_hash_encadenamiento.cpp

Sin rehash frente a con rehash

n	α fijo	comparaciones	α	m	comparaciones
1 000	0,99	1,48	0,99	1 009	1,48
10 000	9,91	5,95	0,62	16 249	1,30
100 000	99,1	50,6	0,77	130 027	1,38
400 000	396	**199**	0,77	520 193	**1,39**

Izquierda: m = 1009 fijo. Derecha: rehash al doble cuando α pasaría de 1.

El rehash, amortizado (400 000 inserciones desde m = 7):
nodos recolocados en todos los rehash juntos: 701 662 → **1,75 por inserción**

La cuenta del array dinámico del tema 2: $1 + ½ + ¼ + \ldots < 2$.
Cada rehash es O(n), pero insertar sigue siendo **O(1) amortizado**.

03_hash_encadenamiento.cpp

Direccionamiento abierto

Todo dentro del array

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Direccionamiento abierto: sondeo lineal

El encadenamiento hace un **new por clave**, y cada salto de nodo en nodo es un salto por memoria que la caché no predice.

Alternativa: **sin listas**. Si la casilla h(c) está ocupada, probar la siguiente…

casilla del intento k = (h(c) + k) mod m, k = 0, 1, 2…

Buscar se para en la primera casilla **VACÍA**: "si estuviera, estaría antes".

El precio: **$\alpha < 1$** siempre (no caben más claves que casillas),
y el coste se dispara al acercarse a 1. Se redimensiona en $\alpha = 0{,}7$.

04_hash_direccionamiento_abierto.cpp

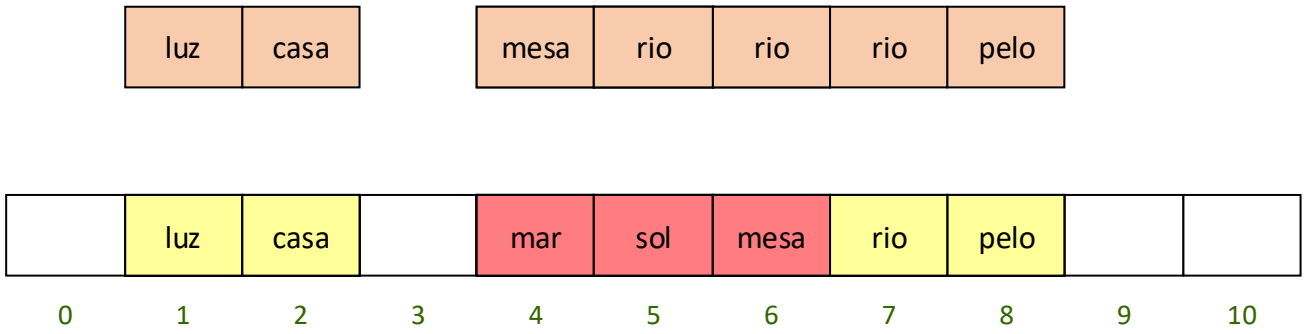
Implementación

Direccionamiento abierto

```
int i = hash(c) % m;
while (casillas_[i].estado== OCUPADA) {
   // (y si es c: sustituir)
   i = (i + 1) % m;
}
casillas_[i] = {c, v, OCUPADA};
n_++;
```

Las claves que chocan forman BLOQUES contiguos, y cualquier clave que caiga en un bloque lo alarga: AGRUPAMIENTO primario.

insertar: sondeo lineal



mesa una

Implementación

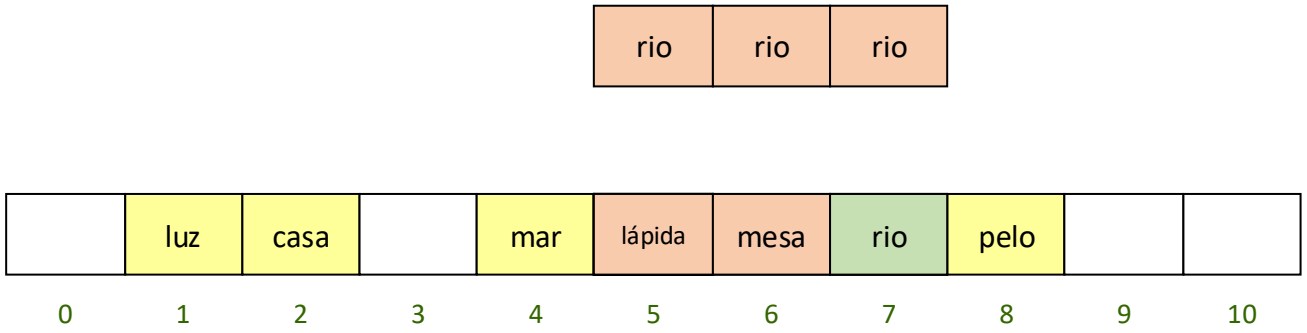
Direccionamiento abierto

```
bool borrar(const string& c) {
   int pos = posicionDe(c);
   if (pos == -1) return false;
   casillas_[pos].estado = BORRADA;
   n_--;
   lapidas_++;
   return true;
}
```

Una LÁPIDA: para buscar cuenta como ocupada (se salta y se sigue), para insertar como libre (se reutiliza).

Las lápidas alargan los caminos igual que las claves: cuentan en α.

borrar("sol")



...buscar(**rio**) en la tabla!

Ejercicio 4: borrar con lápida

Completa borrar() en 04_hash_direccionamiento_abierto.cpp.

sol y rio empiezan en la misma casilla, y rio entró después.
Si se borra sol dejando su casilla vacía, ¿qué pasa al buscar rio?

Buscar rio empieza en la casilla de sol, la ve vacía y se para: "no está".
Y **sigue en la tabla**. Vaciar corta el camino de todas las claves
que saltaron por encima de esa casilla.

La lápida (BORRADA): buscar la salta; insertar la reutiliza,
después de comprobar que la clave no está más adelante.

04_hash_direccionamiento_abierto.cpp

Sondeos medios frente a α

α	éxito	teoría	fracaso	teoría	encadenamiento
0,25	1,16	1,17	1,39	1,39	1,12
0,50	1,50	1,50	2,54	2,50	1,25
0,70	2,28	2,17	7,28	6,06	1,35
0,80	3,46	3,00	17,97	13,00	1,40
0,90	7,96	5,50	**90,10**	50,50	1,45
0,95	18,64	10,50	**473,93**	200,50	1,48

Teoría: Knuth (1962), con un hash perfecto. A partir de 0,7, la medida se aparta: es el **agrupamiento primario**. Bloques reales en m = 61 con α = 0,5:

```
...#.#...####..###...##########...#.....#..######.#....##.#...
```

04_hash_direccionamiento_abierto.cpp

Encadenamiento frente a direccionamiento abierto

	Encadenamiento	**Direccionamiento abierto**
Las colisiones van…	a una lista, fuera del array	a otra casilla del array
α	puede pasar de 1	siempre < 1 (0,5 – 0,8)
Memoria por clave	clave + puntero + bloque	clave + estado, contiguo
Caché	mala: un salto por nodo	buena: casillas seguidas
Borrar	quitar un nodo	lápidas
Con un hash regular	aguanta	sufre (agrupamiento)

Las lápidas se acumulan: con 100 claves vivas (α real = 0,1) y 2 400 borrados, buscar lo que no está pasa de **1,1 a 15 sondeos**. Por eso cuentan en α.

04_hash_direccionamiento_abierto.cpp

La STL

unordered_map y unordered_set

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

unordered_map: la tabla del fichero 03

Es una tabla hash con **encadenamiento**, y deja ver sus cubetas:

al insertar n =	**bucket_count()**	**load_factor()**
1	2	0,50
3	5	0,60
6	11	0,55
12	23	0,52
…	…	…
1 598	3 203	0,50

Cada fila es un rehash: el doble, redondeado a primo, en cuanto α pasaría de 1, el valor de max_load_factor().

reserve(n) los evita:
21 rehash → 0
65 ms → 44 ms
(2 millones de claves)

std::hash<int>(12345) = 12345: **la identidad**. Todo el reparto lo hace el mod m primo.

05_unordered_stl.cpp

Lo que hay que saber de unordered_map

operator[] inserta si la clave no está:

```
int e = edad["pepe"];      // ¡pepe se cuela, con valor 0!
```

Para consultar: **find()** o **count()**. at() lanza una excepción.

El orden de recorrido no significa nada, y cambia al crecer la tabla.

Una **clave propia** necesita dos cosas: una **función hash** y **operator==**.
Y si la función hash reparte mal, la tabla es una lista:

Puntos (función hash)	**HashPunto (ms)**	**HashPésimo (ms)**
2 500	0,07	3,43
10 000	0,34	70,79
19 600	0,53	**280,58**

El O(1) es **esperado**.
El peor caso: O(n).

05_unordered_stl.cpp